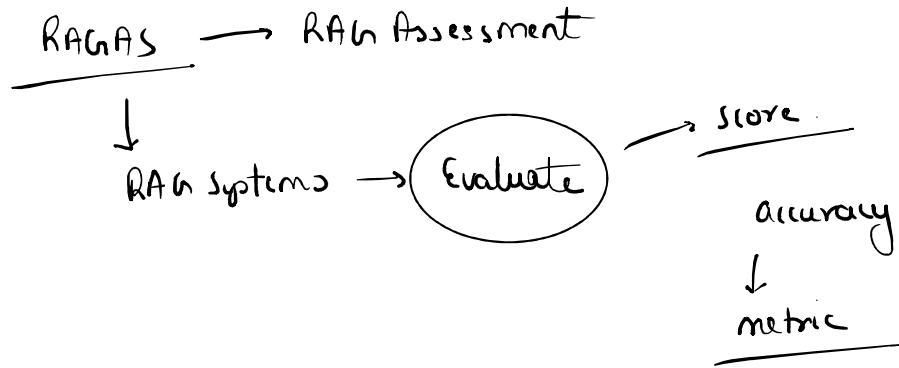




① Ingestion Phase

Accuracy ↓

a) knowledge source → Raw Docs.

✓ b) Chunking → Experimentation.

Strategy

Parameters

chunk overlap
chunk size

✓ c) Embeddings → Embedding model.

Provider

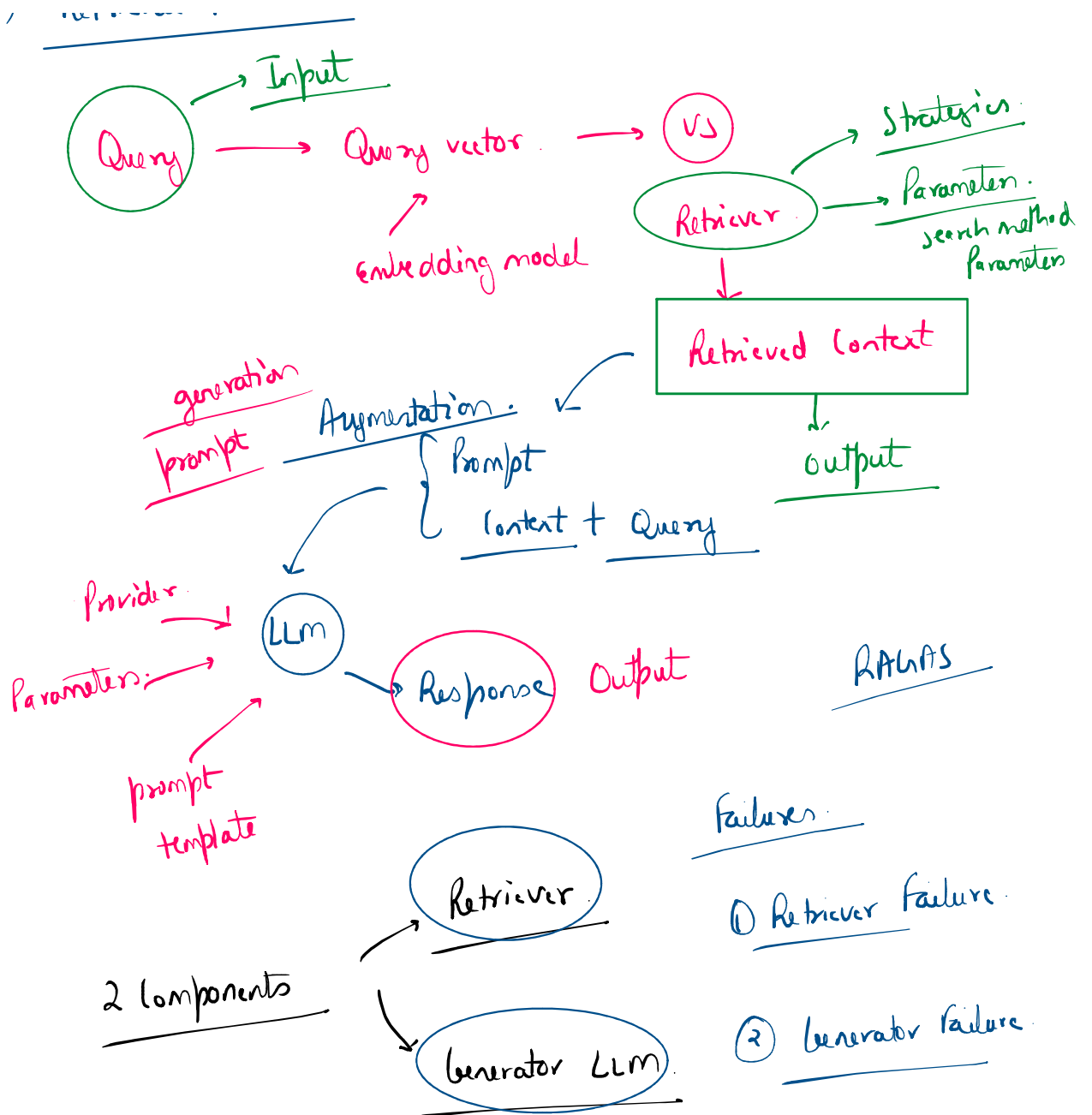
Parameters.

① output dimensions

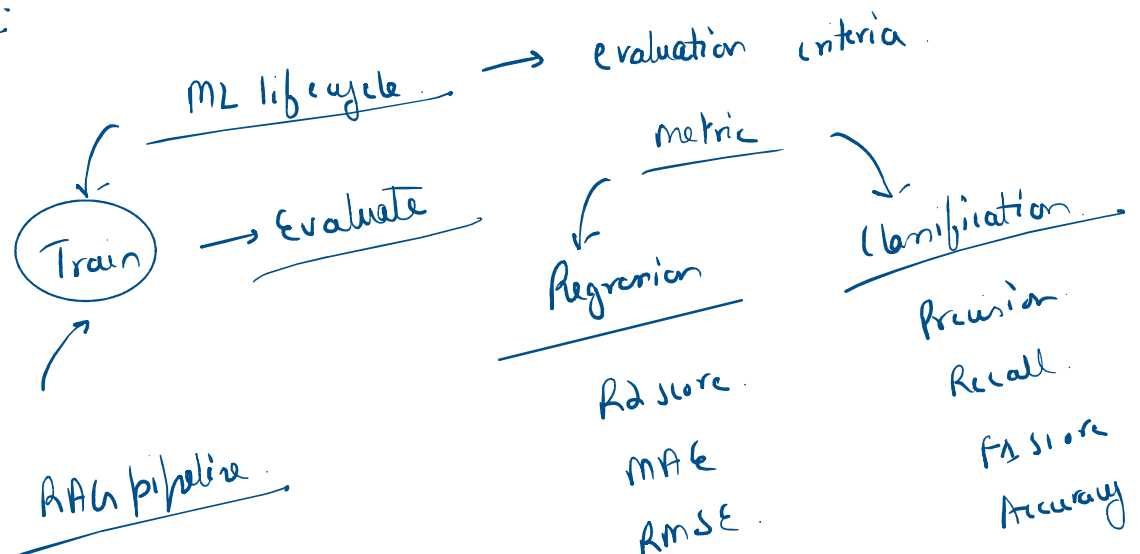
d) vector Database → Provider.

Indexing

2) Retrieval Phase



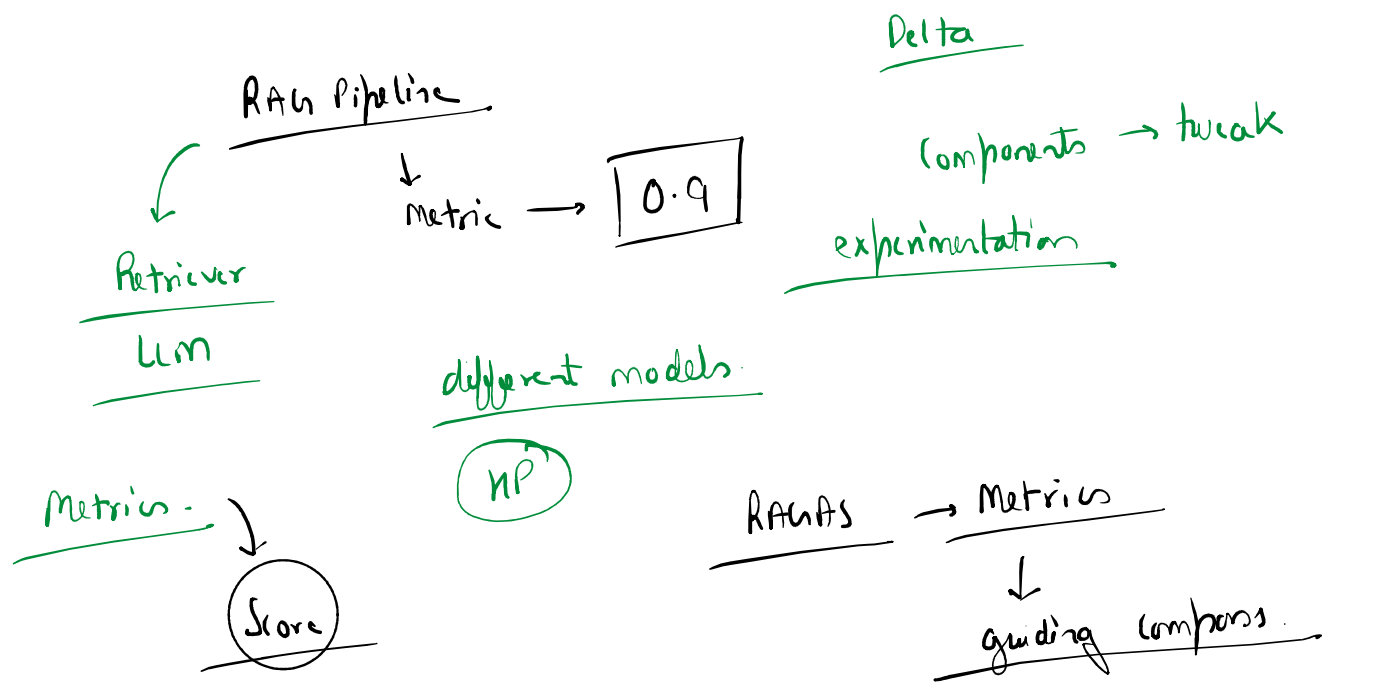
Metric:



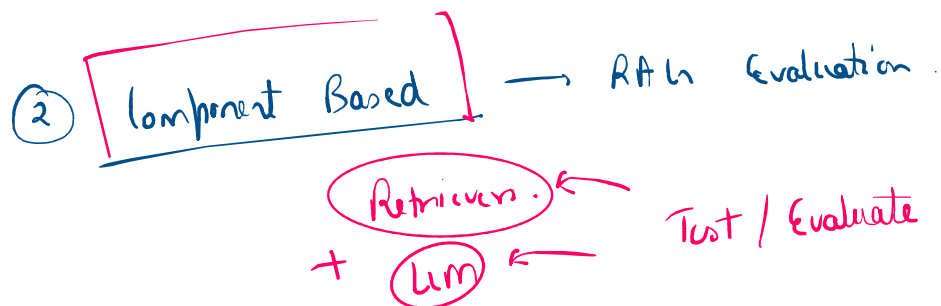
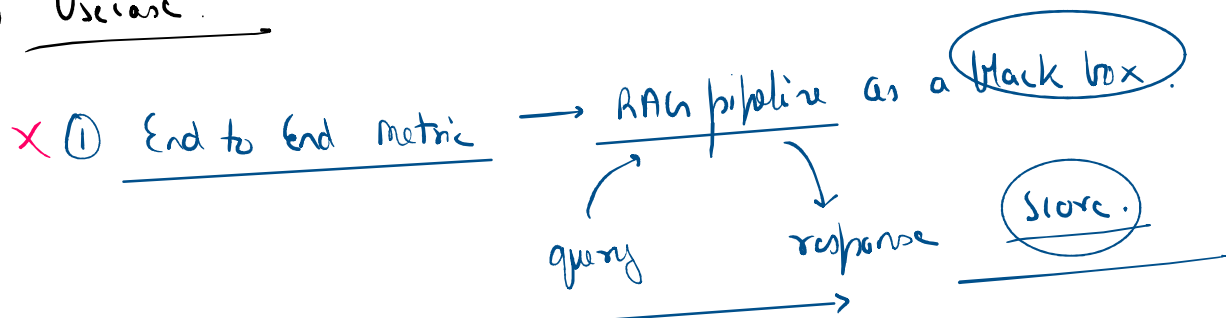
① Retriever →

② LLM →

Why Metrics Matter



① Use case



③ Business Metrics → ✗

① Retriever only

③ Retriever + LLM

Distance

① Retriever only.

③ Retriever + LLM

② LLM only

Context → Response

Output type

① Discrete output based. → pass, fail category

② Numerical output based → int, float range

↓
statistical analysis,
operations

③ Ranked output based → Ranked list

↓
multiple elements → Ranked

RAGAS → Agentic systems

Turns

① Single turn.

end
Input → Processing → Output

RAG Pipeline → Query

↓
Response

② Multi turn

↓
Agentic systems

Loop

conversational

Agentic systems - way
ReAct loop

RAG Pipeline → component Based
→ Single turn metrics

RAG pipeline
quality matters over quantity

Lims

① Llm Based Metrics
② Non Llm Based Metrics → Accuracy
(Llm)

① LLM Based

② Non LLM Based

RAG pipeline.

LLM as a Judge

Response → Capital of India is Mumbai

Grounded Response → Capital of India is New Delhi

① Retriever → Retrieved docs
② LLM → Response.

Answer Key

Retriever → correct Retrieved docs

Comparison

LLM → correct Response

LLM

Keyword matching

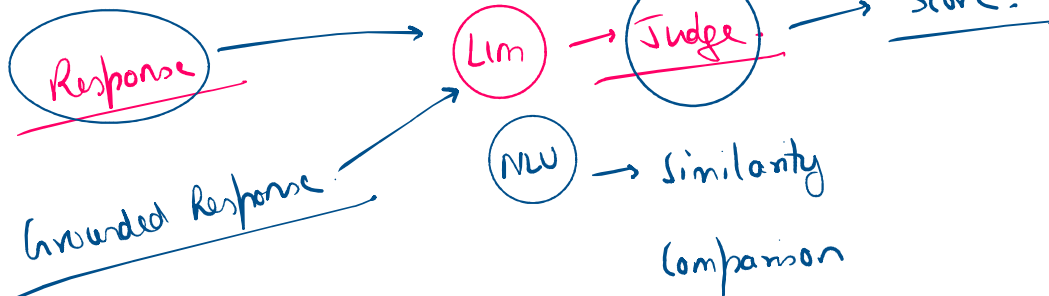
Semantic Similarity

RAG Pipeline

Semantic Meaning

↓
LLM

LLM Based Metrics



Response

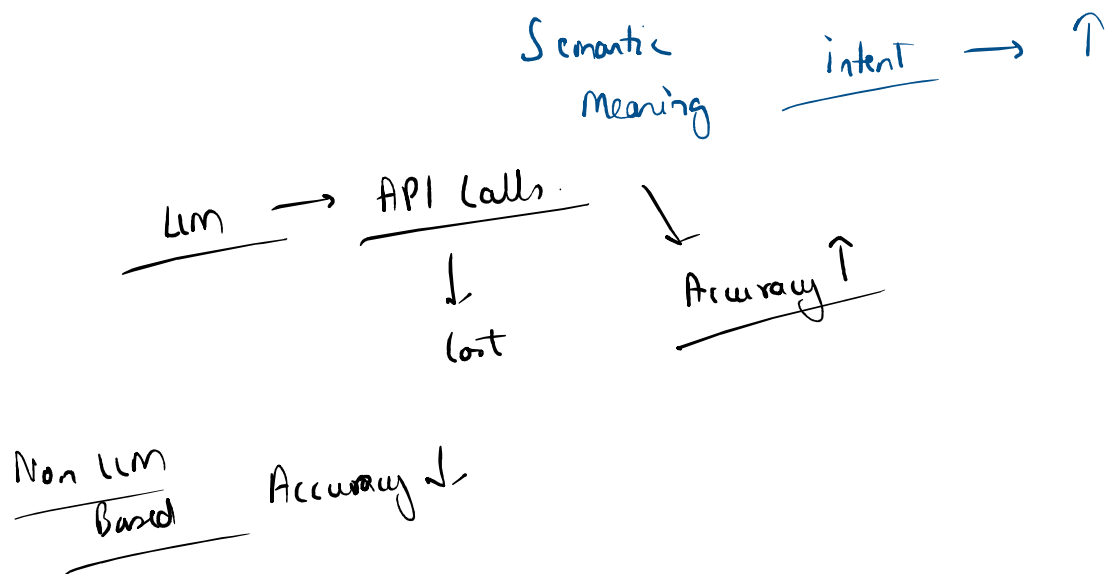
Context }
Query }

LLM

Comparison

Semantic

intent → ↑



Key Terminology

1. sample
2. user_input
3. retrieved_contexts
4. response
5. reference
6. reference_contexts
7. Evaluation Dataset
8. Experiments

① sample → 1 Row, test query → RAG pipeline

② user_input → query

③ retrieved_context → [str]

④ response → RAG Pipeline → output

⑤ reference → Answer Key Grounded Response

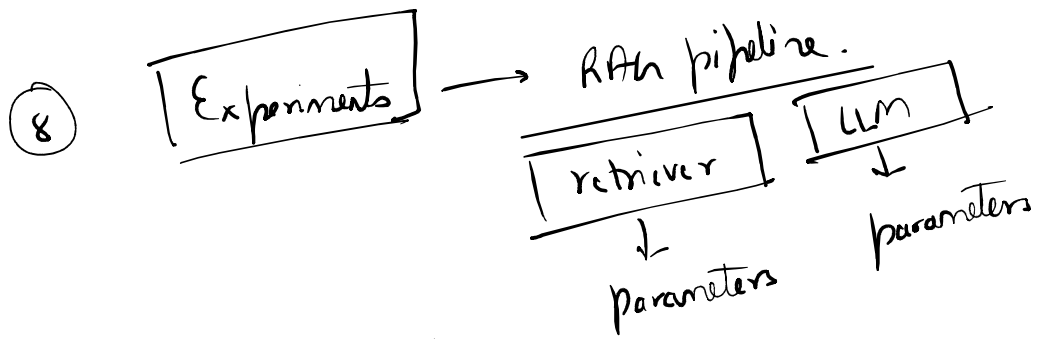
⑥ reference_contexts → retriever

Grounded context

⑦ Evaluation Dataset → RAG pipeline evaluate

train data
└─ train set
└─ val set

train set (val set)



retriever → sim search
k: 3 → 5

LLM → temp = 0.7

What do these metrics target

① Retriever

② Lm

a) Retrieved context → Relevant ^{input}

b) Retrieved context → fact + noise.

Lm

a) Response → grounded → hallucinations.

b) Response → query → useful to answer the query.

c) Response → Robust

a) Context Recall

b) Context Precision

c) Noise sensitivity

d) Response Relevance

e) Faithfulness

① What are these metrics used for?

- ② step by step procedure to calculate
- ③ formula.
- ④ Results $\rightarrow$ score $\rightarrow$ interpret
- ⑤ components

Component based

① Retriever.

② Lm

RetrieverQuery → AI Agents
5 chunks

Context Recall evaluates whether the retriever fetches all the information needed to answer the question correctly. It checks for completeness — did the retriever miss anything important?

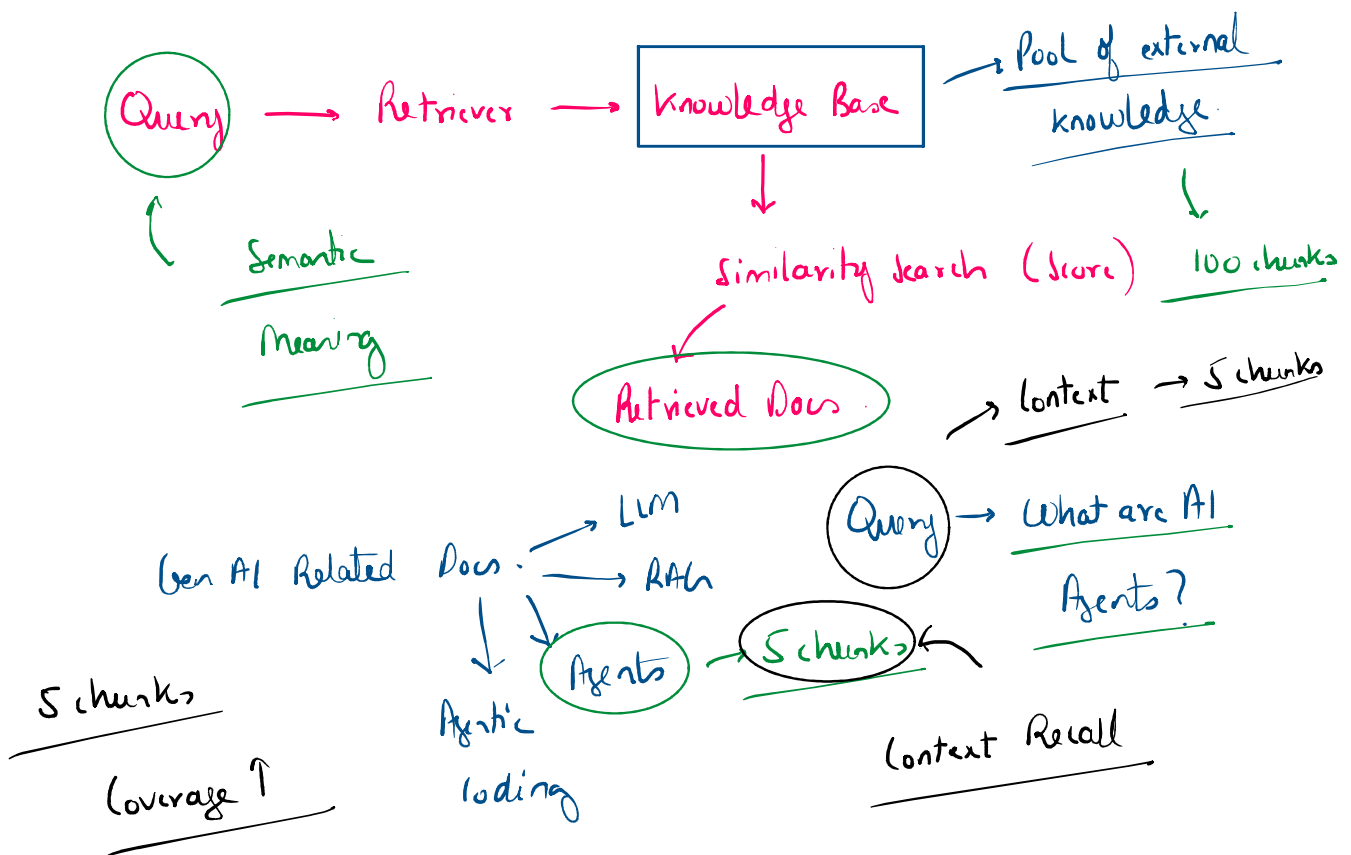
Context Recall measures how many of the relevant documents (or pieces of information) were successfully retrieved. It focuses on not missing important results.

Count

query → context

$$\frac{2}{4} = 0.5$$

$$\text{Context Recall} = \frac{\text{Number of claims in reference attributed to retrieved context}}{\text{Total number of claims in reference}}$$



Your question is: "Tell me about the Eiffel Tower." → Query.

Reference answer (ground truth): "The Eiffel Tower is located in Paris. It was built in 1889. It is 330 metres tall. It was designed by Gustave Eiffel."

Step 1 — Break the reference into individual claims:

Claim #	Claim
C1 ✓	The Eiffel Tower is located in Paris
C2 ✓	It was built in 1889
C3	It is 330 metres tall ✗
C4	It was designed by Gustave Eiffel ✗

→ Perfect response

Semantic meaning

Lm

API call.

- Chunk 1: "The Eiffel Tower stands in the heart of Paris, France." ✓
- Chunk 2: "Construction of the Eiffel Tower was completed in 1889 for the World's Fair." ✓
- Chunk 3: "France is home to many famous landmarks including the Louvre museum."

Retrieved context

Lm

Lm as a Judge → Lm based techniques.

$$\text{Recall} = \frac{2}{4} = 0.5$$

① Query → User AI Agents

② Grounded Response → Generation Response → Perfect Response

Grounded Response → Reference → Facts list down Answer the Query.

a) Lm to extract all the imp facts from the grounded response. → 10 facts AI Agents

→ Retriever

b) Compare the facts extracted with my retrieved context

60% 6 facts Recall = $\frac{6}{10} = 0.6$

40% X

$\frac{1 \rightarrow 0}{0 \rightarrow 1}$ (FP) X

ML $\rightarrow$ Recall $\rightarrow \frac{TP}{TP + FN}$ $\rightarrow$ $\frac{\text{All positive rows.}}{\text{Actual positives.}}$

Coverage

Negative

K: 10

5 chunks
Recall $\uparrow$

5 chunks AI Agents
3 chunks LLMs X
2 chunks RAG X

Recall $\downarrow$
3 chunks AI Agent
5 chunks LLM
2 chunks RAG

Query $\rightarrow$ Related chunks.

Recall $\downarrow$ $\rightarrow$ [0 - 1]

What a low score tells you:

- Your retriever is missing important chunks from the vector store
- The LLM will be forced to either hallucinate the missing facts or give an incomplete answer — through no fault of its own
- Possible causes: chunk size too small, top-k too low, embedding model not capturing semantic meaning well enough

Query

Retriever

What a high score tells you:

- Your retriever is successfully fetching all the information needed to construct a complete answer
- The LLM has everything it needs — whether it uses it well is then measured by the generation metrics

Coverage $\uparrow$

Context $\rightarrow$ Coverage $\uparrow$

Retriever

Complex Retriever → ① Relevant
② Ranked higher → query

Context Precision evaluates whether the **relevant chunks are ranked higher** in the retrieved context list. It is not just about whether the right chunks were retrieved — it is about where they appear in the list.

Relevancy, order

Context Precision is a metric that evaluates the retriever's ability to rank relevant chunks higher than irrelevant ones for a given query in the retrieved context. Specifically, it assesses the degree to which relevant chunks in the retrieved context are placed at the top of the ranking.

① Relevant chunks
② Ranking

It is calculated as the mean of the precision@k for each chunk in the context. Precision@k is the ratio of the number of relevant chunks at rank k to the total number of chunks at rank k.

Kth Position → ⑤

Relevant

③
5th

$\frac{3}{5}$

$$\text{Context Precision@K} = \frac{\sum_{k=1}^K (\text{Precision@k} \times v_k)}{\text{Total number of relevant chunks in top K}}$$

Where:

$$\text{Precision@k} = \frac{\text{true positives@k}}{\text{true positives@k} + \text{false positives@k}}$$

Summation 1 to k → Precision - Relevancy factor

top 5

$$\frac{TP}{TP + FP}$$

ML

① Relevant → Query → Useful

K=10 → ⑤

② Rank higher in the list

Top 5 → ↑

Relevant context → Similarity search

Ranking → High Rank

Context Precision

① Query → Relevant

② Retrieved context

Query ← Relevant to answer the query
Retrieved context

⑤ →

$v_k = \text{Boolean value}$

0.1 1 1 1 1

Query + C₁
↓

V_k = Boolean value
Relevancy factor

Query 1 (1)
↓
LLM

Retriever → $k=5$

Query → "What is the capital of France?"

Query
+
(1)
↓
LLM

Position (k)	Chunk	Relevant? (v_k)
1	"Paris is the capital of France."	1 True
2	"France is famous for its cuisine."	0 Irrelevant
3	"The Eiffel Tower is in Paris."	1
4	"France joined the EU in 1993."	0
5	"Paris has a population of 2 million."	1

Position (k)	Chunk	Relevant? (v_k)
1	"Paris is the capital of France."	1
2	"The Eiffel Tower is in Paris."	1
3	"Paris has a population of 2 million."	1
4	"France is famous for its cuisine."	0
5	"France joined the EU in 1993."	0

What a low score tells you:

- Your retriever is fetching relevant chunks but burying them behind irrelevant ones
- The LLM ends up seeing noise before signal — which can hurt the quality of the generated answer
- You may need to tune your similarity search, re-ranking strategy, or embedding model

What a high score tells you:

- Your retriever is correctly prioritizing the most relevant chunks at the top
- The LLM gets the best context first, giving it the best chance to generate a good answer

Similarity
search
ReRanker

Precision @ k → For each of the k values

$k=5$

5 times

calculate that k

$k=5$

5 times

1 upto that k

$$\boxed{\text{Precision@}k \times v_k}$$

$$\frac{TP}{TP + FP}$$

$$v_k \rightarrow \frac{\text{Boolean value}}{1/0}$$

$$\checkmark \text{ Precision@1} = \frac{1}{1} \times 1 = 1$$

Ranked at
higher position
+
Relevant

$$\checkmark \times \text{ Precision@2} = \checkmark \frac{1}{2} \times v_k = \frac{0.5 \times 0}{1} = 0$$

$$\text{Precision@3} = \frac{2}{3} \times 1 = 0.67$$

$$\text{Precision@4} = 0$$

$$\text{Precision@5} = \frac{3}{5} = 0.6$$

$$\checkmark \checkmark \checkmark$$
$$\underline{\text{Context Precision}} = \frac{1 + 0.67 + 0.6}{3}$$

top 5

2 position
4 position

$$= \frac{2.27}{3} \quad 2.27/3=0.7567$$

$$\boxed{\text{Context Precision@5} = \underline{0.756}}$$

→ 1

2 position
4 position

$$\text{Context Precision @ 5} = \underline{0.756}$$

→ 1

Relevant + High Ranking

→ Top positions

Relevant

$$\text{Precision @ 1} = \frac{1}{1} \times 1 = 1$$

$$\text{Precision @ 2} = 1$$

$$= \text{Context Precision @ 5} = \frac{3}{3}$$

$$\text{Precision @ 3} = 1$$

$$\rightarrow = \boxed{1}$$

top positions

Retriever

Precision
Recall } Retriever

5 Metrics

Robustness of LLM

Noise Sensitivity evaluates how much the LLM is misled by irrelevant chunks in the retrieved context. It measures the robustness of your LLM — does it stay faithful to the relevant chunks, or does it get distracted by the noise and start making incorrect claims?

$$\text{Noise Sensitivity} = \frac{\text{Number of incorrect claims in response attributed to noise}}{\text{Total number of claims in response}}$$

② No. of noisy claims. Response.

0.5

④ LLM → Noisy info

Query

Your question is: "When was the Eiffel Tower built and how tall is it?"

Reference answer (ground truth): "The Eiffel Tower was built in 1889 and is 330 metres tall."

Step 1 — Your retriever fetches these chunks:

Chunk	Type
Chunk 1: "The Eiffel Tower was completed in 1889."	✓ Relevant
Chunk 2: "The Eiffel Tower stands 330 metres tall."	✓ Relevant
Chunk 3: "The Louvre Museum attracts 9 million visitors per year."	✗ Noise
Chunk 4: "The Arc de Triomphe is 50 metres tall."	✗ Noise

Facts extracts

Built in 1889

330 m tall

Useful
Noisy.

Retrieved
chunks

Response
Generated by LLM

"The Eiffel Tower was built in 1889. It stands 330 metres tall. It attracts 9 million visitors per year. It is 50 metres tall."

Reference /
Grounded Response → without noise.

"The Eiffel Tower was built in 1889 and stands 330 metres tall."

$$NS = \frac{2}{4} = 0.5$$

① Query

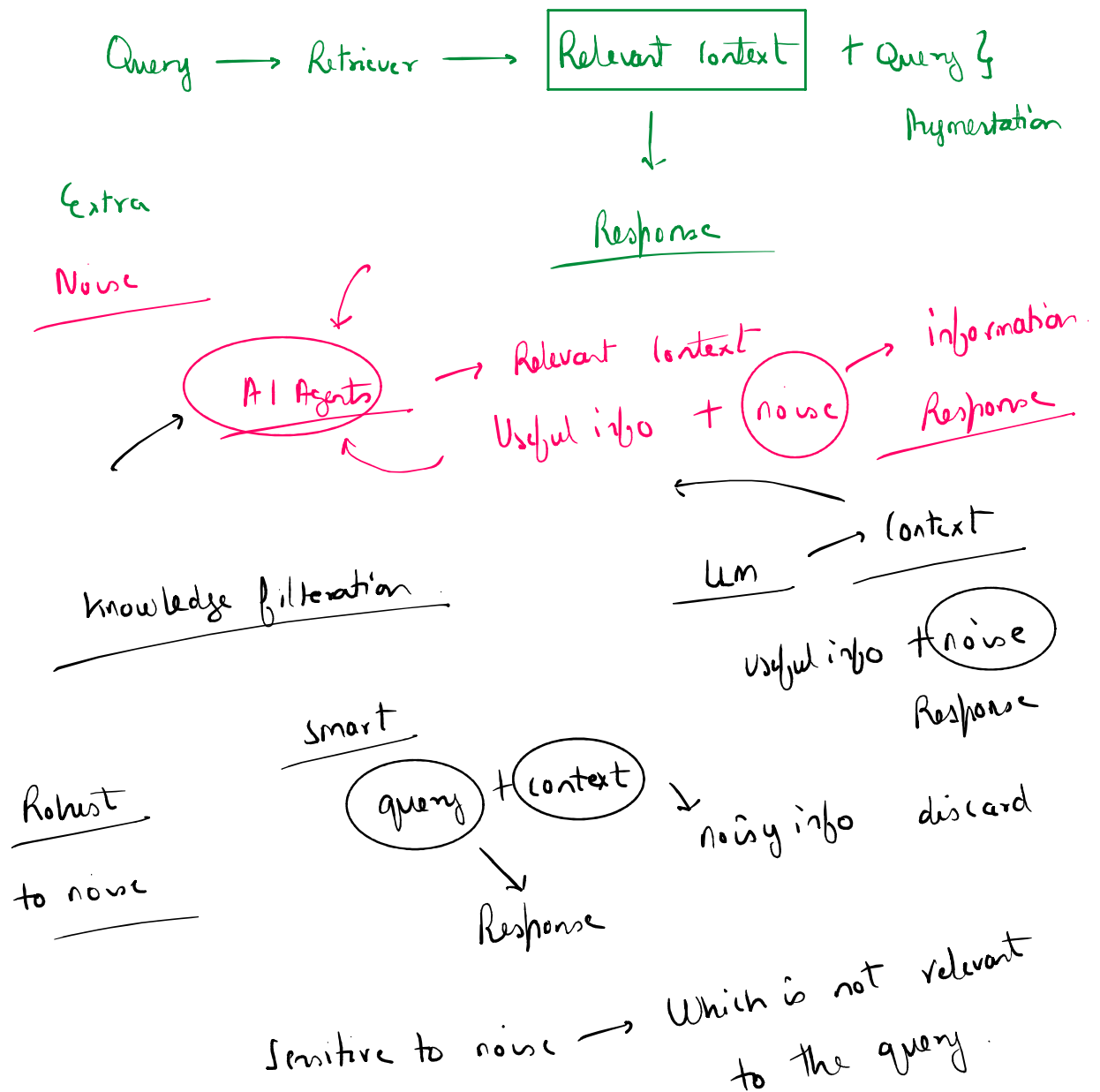
② Grounded Response / Reference → Actual Answer → facts those

one useful facts only.

↓
without noise

③ Response → generated by Llm

④ Retrieved context → chunks



Steps

- ① Grounded Response $\rightarrow$ LLM $\rightarrow$ Facts / Claims $\rightarrow$ extract imp details
- ② Retrieved Docs $\rightarrow$ Match $\rightarrow$ Useful info
Not Matching $\rightarrow$ Noisy info
- ③ Response $\rightarrow$ Facts $\rightarrow$ count $\rightarrow$ noisy facts

What a high score tells you: Bad.

- Either your retriever is bringing in too many irrelevant chunks, or
- Your LLM is not robust enough to ignore irrelevant context, or
- Both

What a low score (close to 0) tells you: Good.

- Your LLM is successfully distinguishing relevant from irrelevant chunks
- Even when the retriever brings in noise, the LLM is not misled by it

Retriever ✗

LLM $\rightarrow$ query, context, reference, response $\rightarrow$ intent
keyword matching

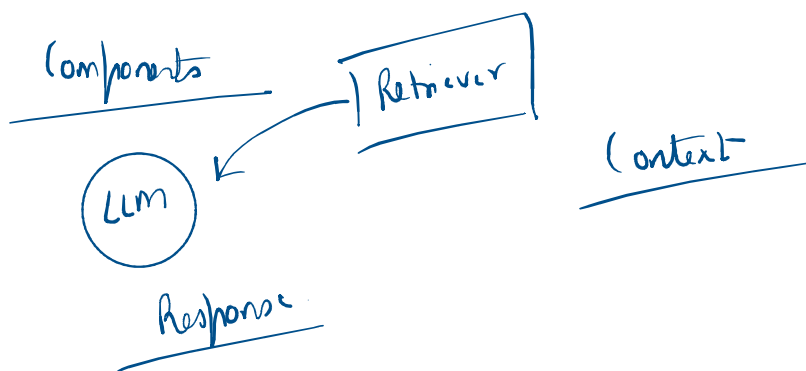
NS $\uparrow$ = Robustness $\downarrow$

lower is Better

0 $\rightarrow$ Noisy info ✗

1 $\rightarrow$ Noisy info ✓

0.5 = Half of response is made up of noisy facts.



Response Relevance

Response → useful to answer the query → factual accuracy

Response Relevancy evaluates whether the **generated response actually addresses the user's question**. It checks if the answer is on-topic and directly relevant to what was asked — not whether it is factually correct.

Factual accuracy

3 queries
N=3

Mean

$\cos(e_g, e_o)$

$e_g \rightarrow$ embedding gen.
 $e_o \rightarrow$ embedding original.

Response

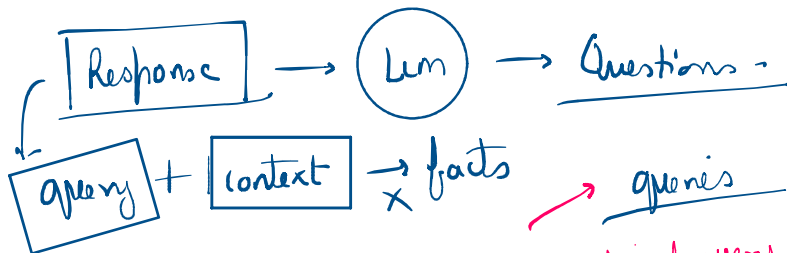
$$\text{Response Relevancy} = \frac{1}{N} \sum_{i=1}^N \cos(e_{g_i}, e_o)$$

Query to Query comparison

$$\cos(e_{q_1}, e_o) + \cos(e_{q_2}, e_o) + \cos(e_{q_3}, e_o)$$

Goal → Response is it useful or not → to answer the query.

Reverse Eng.



① Response → LLM

Original query hypothetical 3 queries

② 3 queries → Embedding model → Embeddings

Similarity → cosine similarity

Original query → vector embedding

Hypothetical queries → embeddings

cosine similarity

similarity score

cosine similarity

User Input

Response

HyDE

cosine similarity

Query → "What is the capital of France?"

Response → "Paris is the capital of France. It is also the largest city in the country."

① Response

LLM

3 scores

(Response) →

"Paris is the capital of France. It is also the largest city in the country."

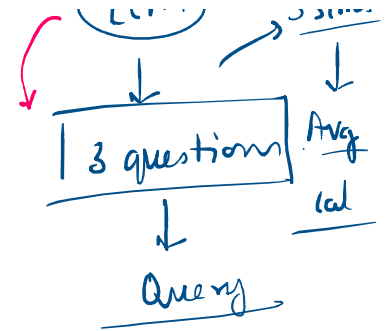
- Q1: "What is the capital city of France?"
- Q2: "Which city serves as France's capital?"
- Q3: "What is the largest and capital city of France?"

↑
Reverse
eng

Not
useful

Useful

Similarity



✗
"France is a country in Western Europe. It is known for its wine, cheese, and the Eiffel Tower. France has a population of about 68 million people." ✓

- Q1: "What is France known for?"
- Q2: "Where is France located?"
- Q3: "What is the population of France?"

Query → cosine sim

↓ Avg similarity

LLM

What a low score tells you:

- Your LLM is going off-topic or giving evasive answers
- Your prompt template may not be instructing the LLM clearly enough to stay focused on the question
- The LLM may be padding responses with unnecessary information

What a high score tells you:

- Your LLM is staying focused and directly addressing what the user asked
- The prompt template is working well in keeping the LLM on track

Factual
Accuracy ✗

Faithfulness

Faithfulness

Facts in response → grounded in retrieved context

Faithfulness evaluates whether every claim made in the generated response can be traced back to the retrieved context. It is the primary hallucination detection metric in RAGAS.

A response is considered faithful if the LLM only says things that are directly supported by the retrieved chunks — it should not bring in outside knowledge, make assumptions, or invent facts that are not present in the context.

↓
hallucinations

Context

LLM → Made up facts → Response → Retrieved context

$$\text{Faithfulness} = \frac{\text{Number of claims in response that are supported by retrieved context}}{\text{Total number of claims in response}}$$

→ True

LLM → extract

① Retrieved context → Retriever

② Response → LLM after reading the context

steps ① Response → Facts extract → LLM

② 5 facts extract → Retrieved context

③ Count the number of True value.

③

Binary label.
True / False

↓
grounded ↓
hallucinating

Query

"Tell me about Albert Einstein."

Retrieved context

Retrieved chunks:

- Chunk 1: "Albert Einstein was born on 14 March 1879 in Ulm, Germany."
- Chunk 2: "Einstein developed the theory of relativity, one of the two pillars of modern physics."
- Chunk 3: "Einstein was awarded the Nobel Prize in Physics in 1921."

3 chunks

Response

LLM

key facts

"Albert Einstein was born on 14 March 1879 in Germany. He developed the theory of relativity. He won the Nobel Prize in Physics in 1921. He was also known for his work on quantum mechanics. He had an IQ of 160."

Response

Context

+

Query

Iterate
one by one

LLM

Claim #	Claim	
C1	Born on 14 March 1879 in Germany	✓ True
C2	Developed the theory of relativity	✓ True
C3	Won the Nobel Prize in Physics in 1921	✓ True.
C4	Known for work on quantum mechanics	✓ False.
C5	Had an IQ of 160	✓ False

Grounded/
Retrieved
context

Each chunk/
fact
retrieved
context

$$= \frac{3}{5}$$

Faithfulness
= 0.6

Context

What a low score tells you:

- Your LLM is hallucinating — making claims beyond what the retrieved context supports
- Your prompt template may not be clearly instructing the LLM to only use the provided context
- The LLM may be over-relying on its own parametric knowledge instead of the retrieved chunks

What a high score tells you:

- Your LLM is staying strictly within the boundaries of the retrieved context
- Hallucination is under control

[0-1]

LLM

Output